

实验名称 实验4 复杂网络的社团检测

一、实验内容及结果

- 1、给定网络 1tux400.csv，采用 3 种算法进行社团检测，输出模块度并对社团划分后的网络进行可视化；基于社团检测结果，分析这 3 种算法的特点。
- 2、对教材 P144 图 4-19 的 a 网络，参考 networkx-reference3.0.pdf，采用派系过滤算法进行社团检测。

源代码：

Task1:

引入网络

```
: import networkx as nx
import networkx.algorithms.community as nx_comm
import matplotlib.pyplot as plt
import pandas as pd
from random import randint
import numpy as np

df= pd.read_csv("D:/Netdata/ex3/1tux400.csv")
G=nx.from_pandas_edgelist(df,'source','target','weight',create_using=nx.Graph())
nx.draw(G, node_size=50, with_labels=True)
plt.show()
```

Louvain:

```
partition=nx_comm.louvain_communities(G)
print("网络社团为: ",partition)

Q=nx_comm.modularity(G,partition)
print("模块度Q=",Q)

colors= ["" for x in range(G.number_of_nodes())]
counter=0

node_to_int = {node: idx for idx, node in enumerate(G.nodes())}

for com in partition:
    color="#%06X" % randint(0,0xFFFFFF)
    counter+=1
    for node in list(
        com
    ):
        colors[node_to_int[node]] = color

plt.figure(figsize=(12,10))
plt.axis("off")
nx.draw_networkx(G,node_size=50,with_labels=False,width=0.5,node_color=colors)
```

Givan-Newman:

```
#GN 运行时间长
from networkx.algorithms import community
import itertools
k=10
comp= community.girvan_newman(G)
limited= itertools.takewhile(lambda c: len(c) <=k, comp)
for communities in limited:
    b=list(sorted(c) for c in communities)
print(b)
```

```
colors= [" " for x in range(G.number_of_nodes())]
counter=0

node_to_int = {node: idx for idx, node in enumerate(G.nodes())}

for com in b:
    color="#%06X" % randint(0,0xFFFFFF)
    counter+=1
    for node in list(
        com
    ):
        colors[node_to_int[node]] = color
counter

plt.figure(figsize=(12,10))
plt.axis("off")
nx.draw_networkx(G,node_size=50,with_labels=False,width=0.5,node_color=colors)
```

Kernighan-Lin:

```
from networkx.algorithms.community import kernighan_lin_bisection
P=kernighan_lin_bisection(G)
print("网络社团为: ",P)

Q=nx_comm.modularity(G,P)
print("模块度Q=",Q)
```

```
colors= [" " for x in range(G.number_of_nodes())]
counter=0

node_to_int = {node: idx for idx, node in enumerate(G.nodes())}

for com in partition:
    color="#%06X" % randint(0,0xFFFFFF)
    counter+=1
    for node in list(
        com
    ):
        colors[node_to_int[node]] = color

plt.figure(figsize=(12,10))
plt.axis("off")
nx.draw_networkx(G,node_size=50,with_labels=False,width=0.5,node_color=colors)
```

Task2:

```
colors = [""] for x in range(G.number_of_nodes())
counter=0

node_to_int = {node: idx for idx, node in enumerate(G.nodes())}

for com in partition:
    color="#%06X" % randint(0,0xFFFFFF)
    counter+=1
    for node in list(
        com
    ):
        colors[node_to_int[node]] = color

plt.figure(figsize=(12,10))
plt.axis("off")
nx.draw_networkx(G,node_size=50,with_labels=False,width=0.5,node_color=colors)
```

```
def get_percolated_cliques(G, k):
    cliques = list(frozenset(c) for c in nx.find_cliques(G) if len(c) >= k) # 找出所有大于k的最大k-派系

    matrix = np.zeros((len(cliques), len(cliques))) # 构造全0的重叠矩阵

    for i in range(len(cliques)):
        for j in range(len(cliques)):
            if i == j: # 将对角线值大于等于k的值设为1
                n = len(cliques[i])
                if n >= k:
                    matrix[i][j] = 1
            else:
                matrix[i][j] = 0
        else: # 将非对角线值大于等于k的值设为1
            n = len(cliques[i].intersection(cliques[j]))
            if n >= k - 1:
                matrix[i][j] = 1
            else:
                matrix[i][j] = 0

    l = list(range(len(cliques)))
    for i in range(len(matrix)):
        for j in range(len(matrix)):
            if matrix[i][j] == 1 and i != j:
                l[j] = l[i]

    q = [] # 社区号
    for i in l:
        if i not in q:
            q.append(i)

    p = [] # 所有社区
    for i in q:
        print(frozenset.union(*[cliques[j] for j in range(len(l)) if l[j] == i]))
        p.append(list(frozenset.union(*[cliques[j] for j in range(len(l)) if l[j] == i])))

    return p
```

```

communities=get_percolated_cliques(G, 4)

plt.figure(figsize=(10, 5))
for i, community in enumerate(communities):
    subgraph = G.subgraph(community)
    plt.subplot(1, len(communities), i + 1)
    nx.draw(subgraph, pos,with_labels=True, node_color="red", node_size=500)
    plt.title(f"Community {i + 1}")

plt.tight_layout()
plt.show()

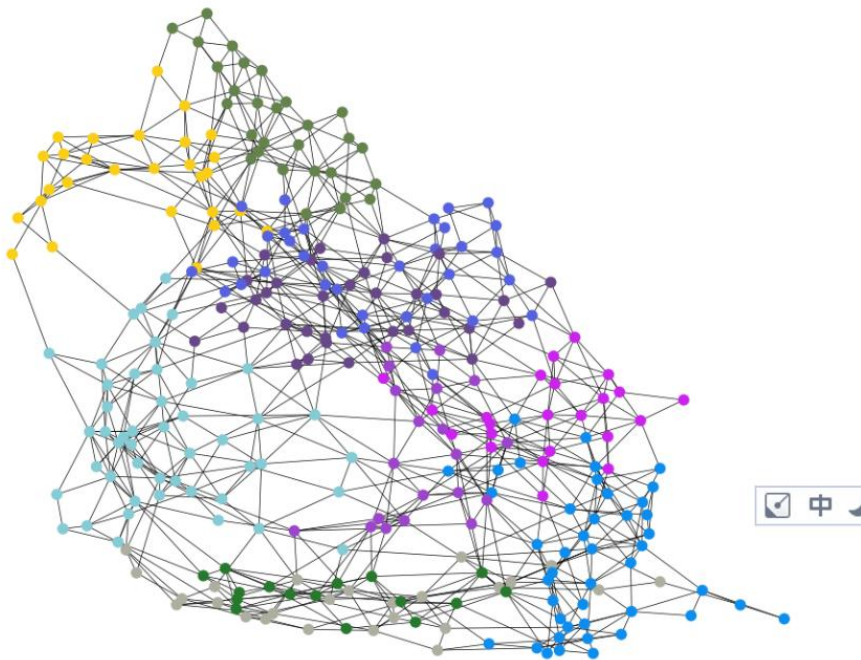
```

运行结果（截图）：

Task1:

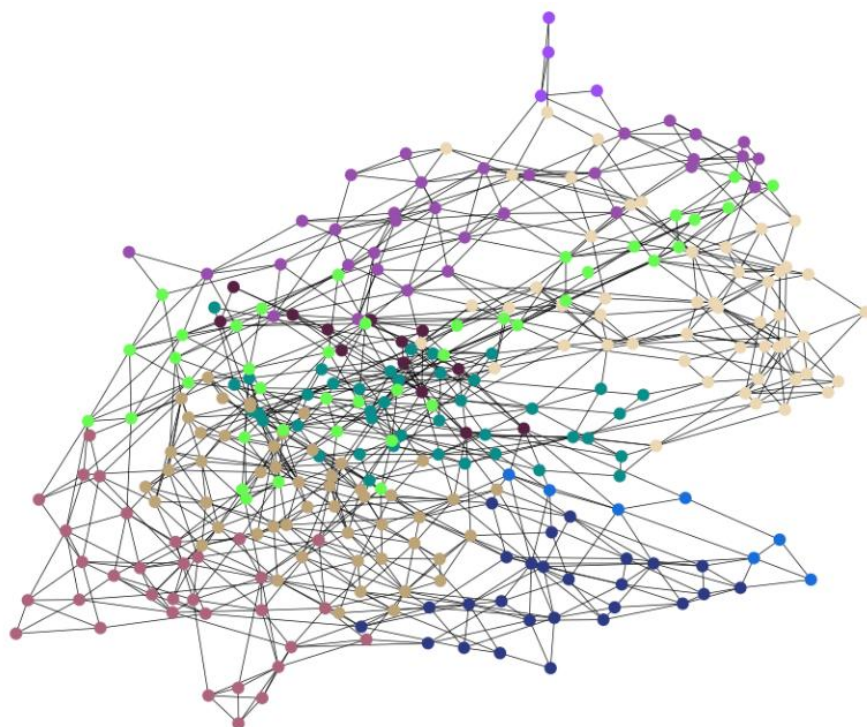
Louvain:

模块度Q= 0.7134693486681765



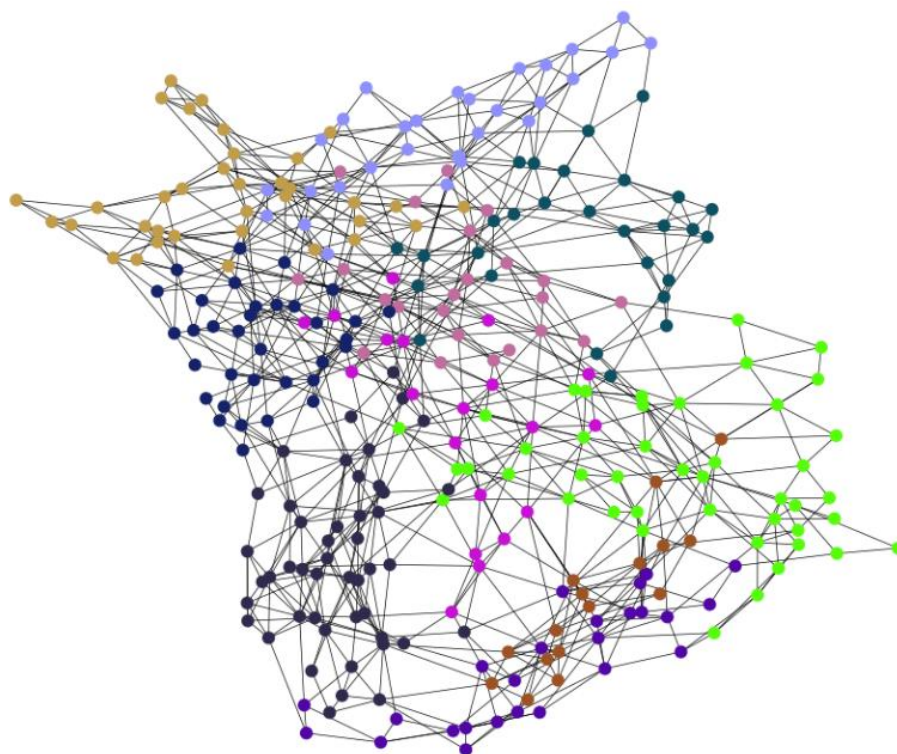
Givan-Newman:

模块度Q= 0.651276089423459



Kernighan-Lin:

模块度Q= 0.4124449722047987



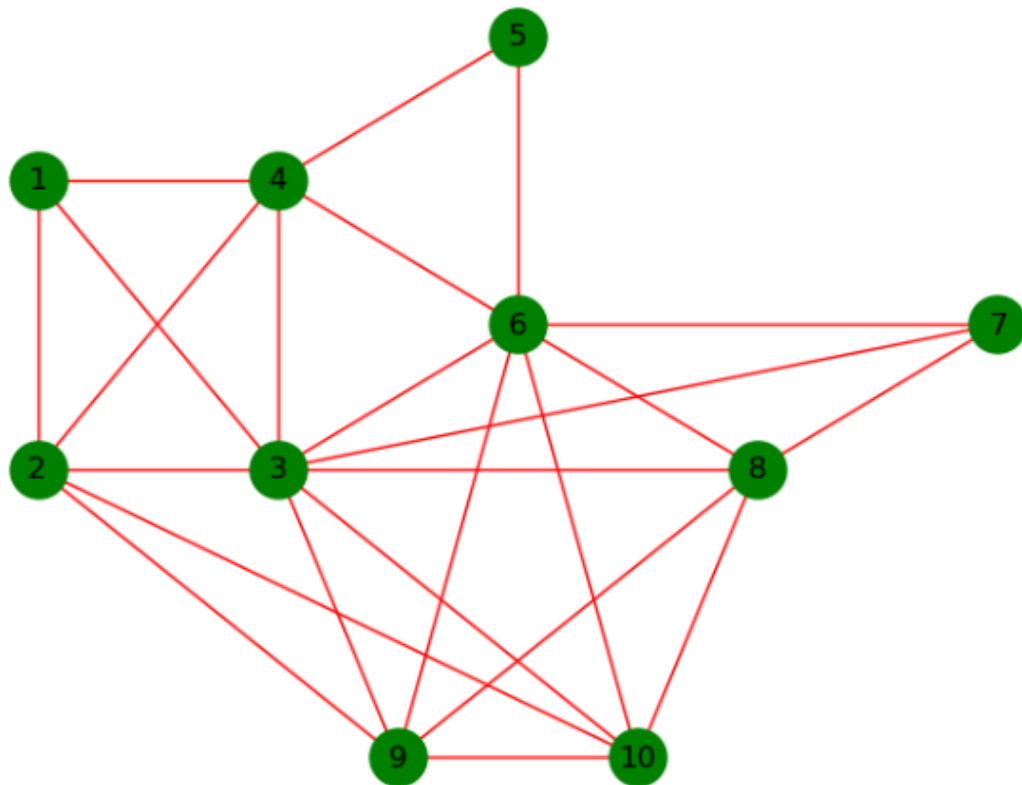
对于 1tux400 网络，我使用了三种不同的算法进行社团检测并计算了模块度，分别是 Louvain 算法、Givan-Newman 算法、Kernighan-Lin 算法。其中，Louvain 所得模块度最高，达到 0.71，Louvain 算法是一种基于贪心策略的社团检测方法，它的特点是不断优化模块度来发现网络中的社团结构，所以模块度高，社团划分质量较高。

Givan-Newman 算法模块度稍低，为 0.65。Givan-Newman 算法是基于层次聚类的社团检测方法，它通过迭代删除网络中的边来发现社团结构，能够较好地社团结构。且实验时发现运行 GN 方法代码时间长，表明 GN 方法算法复杂度高。

Kernighan-Lin 算法分隔得到社团的模块度最低，为 0.41。Kernighan-Lin 算法是一种基于图分割的方法，它通过迭代交换节点来改进社团划分的质量，由于 Kernighan-Lin 算法在每次迭代中只进行局部搜索，因此可能会陷入局部最优解导致模块度较低。

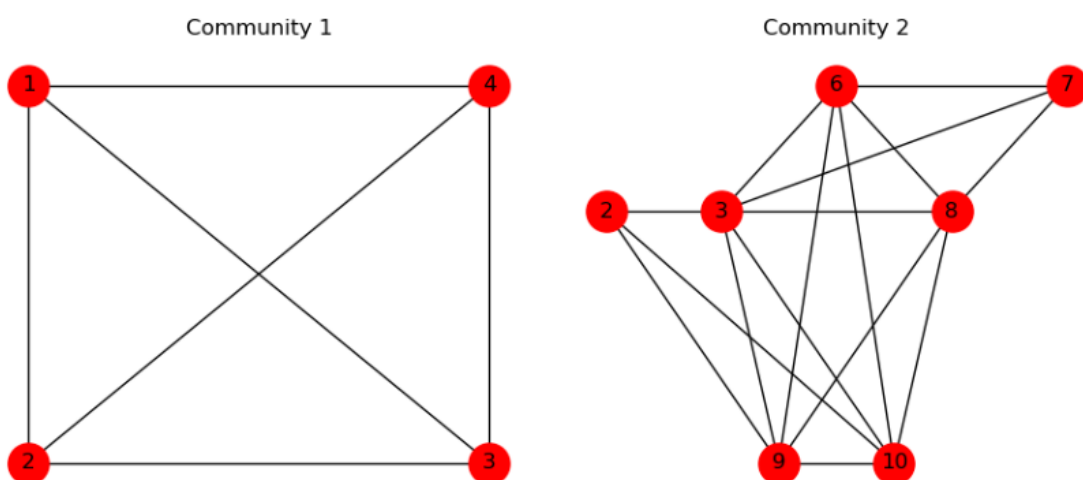
Task2:

教材 P144 图 4-19 的 a 网络:



派系过滤社团检测结果:

```
frozenset({1, 2, 3, 4})  
frozenset({2, 3, 6, 7, 8, 9, 10})  
[[1, 2, 3, 4], [2, 3, 6, 7, 8, 9, 10]]
```



由派系过滤结果可知，a 网络被划分为两个社团。其中，社团 1 由节点 1234 构成；社团 2 由节点 236789 10 构成。